

MACHINE LEARNING PROJECT — PHASE 2 PROPOSAL

Crowd Density Image Classification Framework

A CNN-Based Decision-Support System for

Public Safety and Crowd Monitoring

Submitted by: Nithin Chowdary Raya

Student ID: 69683173

Course: Machine Learning

Instructor: Raja Hashim Ali

Date: June 2026

Contents

1	Project Title	3
2	Background and Motivation	3
3	Problem Statement	3
4	Selected Dataset and Dataset Links	4
5	Dataset Description	4
5.1	Dataset 1 — ShanghaiTech	4
5.2	Dataset 2 — JHU-Crowd++	5
5.3	Dataset 3 — Abnormal High-Density Crowds	5
5.4	Combined Class Distribution	6
6	Research Questions	6
7	Expected Results	7
8	Proposed Methodology	7
8.1	Dataset Preparation	7
8.2	Preprocessing Pipeline	8
8.3	Data Augmentation (Training Set Only)	8
8.4	Class Imbalance Handling	8
9	CNN Model Design	8
9.1	Custom CNN Architecture (Baseline)	8
10	Transfer Learning Models	9
10.1	Model 1 — MobileNetV2 (Lightweight)	9
10.2	Model 2 — EfficientNetB3 / EfficientNetV2S (High Accuracy)	9
10.3	Shared Classifier Head (Both TL Models)	10
11	Evaluation Metrics	10
12	Expected Figures and Tables	10
12.1	Figure 1 — Class Distribution	11
12.2	Figure 2 — Custom CNN Training History	11
12.3	Figure 3 — Custom CNN Confusion Matrix	12
12.4	Figure 4 — Sample Images per Density Class	13
12.5	Figure 5 — EfficientNetV2S Training History	14

12.6 Figure 6 — EfficientNetV2S Confusion Matrix	14
12.7 Figure 7 — Custom CNN ROC Curves	15
12.8 Figure 8 — EfficientNetV2S ROC Curves	16
12.9 Figure 9 — Model Comparison	16
12.10 Figure 10 — Grad-CAM Visualisations	17
12.11 Figure 11 — Error Analysis	18
12.12 Table — Actual Model Performance Results	18
12.13 Discussion of Results	19
12.14 Per-Class Classification Reports	19
13 Deployment Discussion, Limitations, and Ethical Concerns	20
13.1 Practical Deployment	20
13.2 Limitations	20
13.3 Ethical Concerns	20
14 Implementation Tools and Environment	21
15 References	21

1 Project Title

Crowd Density Image Classification Framework: A CNN-Based Decision-Support System for Public Safety Monitoring Using Deep Learning and Transfer Learning.

2 Background and Motivation

Crowd-related disasters continue to pose serious public safety risks worldwide. The 2022 Itaewon crowd crush in South Korea resulted in 158 deaths, the 2021 Astroworld Festival stampede caused 10 fatalities, and the 2024 Hathras stampede in India claimed 121 lives. A common factor in all these incidents was the failure to detect dangerously high crowd densities before the situation became fatal. Traditional crowd monitoring depends on manual surveillance, which is prone to delayed response and human error.

Convolutional Neural Networks (CNNs) have demonstrated strong performance in image classification and can be applied to automatically distinguish crowd density levels from camera images. This project develops a CNN-based Crowd Density Image Classification Framework that classifies crowd images into four levels: **Low**, **Medium**, **High**, and **Very High**. The system is designed as a decision-support tool that assists security personnel rather than replacing them, providing automated alerts that can trigger early interventions before dangerous overcrowding escalates.

The applied value of this project comes from solving a practical problem in security and surveillance using publicly available image datasets. The publication strength comes from careful benchmarking, explainability through Grad-CAM, error analysis, and a realistic deployment discussion.

3 Problem Statement

This project addresses the automated multi-class classification of crowd density from still images. Given an input image of a crowd scene, the model must predict one of four density levels: Low, Medium, High, or Very High. This is a supervised image classification problem where density labels are derived from annotated head-point counts in the ground-truth data.

Key challenges include:

1. Significant visual similarity between adjacent density classes, particularly Medium and High
2. Variability in camera angles, lighting conditions, and crowd distribution
3. Limited labeled training data specifically for crowd classification tasks

4. Class boundary ambiguity when converting continuous counts to discrete labels

The system is framed as a screening tool that flags high-density situations for human review, rather than an autonomous decision-maker.

4 Selected Dataset and Dataset Links

Three publicly available crowd image datasets were combined to form a merged training dataset of 7,198 images, significantly improving model generalization compared to using a single dataset.

Table 1: Datasets Used in This Project

Dataset	Kaggle Link	Images Used	Label Source
ShanghaiTech	https://www.kaggle.com/datasets/tthien/shanghaitech	1,198	.mat head count
JHU-Crowd++	https://www.kaggle.com/datasets/hoangxuanviet/jhu-crowd	3,000	YOLO .txt head count
Abnormal High-Density Crowds	https://www.kaggle.com/datasets/angelchi56/abnormal-highdensity-crowds	3,000	Scene type (Very High)
Total		7,198	

5 Dataset Description

5.1 Dataset 1 — ShanghaiTech

ShanghaiTech is the primary and most widely cited crowd counting benchmark dataset. It contains two parts: Part A with dense crowd scenes scraped from the internet, and Part B with sparser crowd images taken from busy streets in Shanghai. Ground-truth annotations are provided as .mat files containing x-y coordinates of every annotated head in each image.

Table 2: ShanghaiTech Dataset Statistics

Property	Part A	Part B	Total
Total Images	482	716	1,198
Training Images	300	400	700
Testing Images	182	316	498
Min Head Count	33	9	9
Max Head Count	3,139	578	3,139
Avg Head Count	501	123	312
Image Type	RGB Color	RGB Color	RGB Color
Annotation Format	.mat	.mat	.mat

5.2 Dataset 2 — JHU-Crowd++

JHU-Crowd++ is a large-scale crowd counting dataset containing 4,372 images with over 1.5 million annotated heads across diverse scenes including indoor venues, outdoor events, and transportation hubs. Annotations are provided in YOLO-format `.txt` files where each line represents one annotated head. The number of lines per annotation file is used as the head count to assign density class labels. Images are split into train, test, and validation subsets. For this project, 3,000 images were sampled to manage GPU memory constraints.

Table 3: JHU-Crowd++ Dataset Statistics

Property	Value
Total Images Available	4,372
Images Used	3,000
Annotation Format	YOLO <code>.txt</code> (one row per head)
Splits Available	Train / Test / Valid
Scene Diversity	Indoor, outdoor, transportation, events
Image Type	RGB Color

5.3 Dataset 3 — Abnormal High-Density Crowds

This dataset contains video frame sequences from four well-known crowd disaster and near-disaster scenarios: Times Square (New York), Las Vegas Strip, Love Parade (Germany), and an Italian stadium. All scenes depict extremely dense crowd conditions and are therefore assigned the **Very High** density label (class 3). The dataset provides important real-world examples of dangerously overcrowded conditions that are critical for

the model to recognize. Frames are organized by scene and viewpoint subfolders. A total of 3,000 frames were sampled from the available 12,201 frames.

Table 4: Abnormal High-Density Crowds Dataset Statistics

Property	Value
Total Frames Available	12,201
Frames Used	3,000
Scenes	Times Square, Las Vegas, Love Parade, Italy
Assigned Density Class	Very High (Label 3)
Annotation Method	Scene-based (all frames = Very High density)
Image Type	RGB Color (video frames)

5.4 Combined Class Distribution

Continuous head counts from ShanghaiTech and JHU-Crowd++ ground-truth annotations are converted into four discrete density classes using quartile-based thresholds. Abnormal High-Density frames are directly assigned class 3.

Table 5: Class Distribution After Label Assignment (Combined Dataset)

Class	Label	Threshold / Assignment	ShanghaiTech	Combined
Low Density	0	$\text{Count} \leq 80$	305	$\sim 1,500$
Medium Density	1	$80 < \text{Count} \leq 163$	294	$\sim 1,600$
High Density	2	$163 < \text{Count} \leq 311$	300	$\sim 1,800$
Very High Density	3	$\text{Count} > 311$ or Abnormal scene	299	$\sim 2,298$
Total			1,198	7,198

6 Research Questions

The following publication-ready research questions guide this project (as specified in the course Idea Book, Idea 50 of 75):

1. **RQ1:** How accurately can CNN-based models perform crowd density classification from images using publicly available image datasets?
2. **RQ2:** Which model family — custom CNN or transfer learning — provides the best balance between predictive performance and computational efficiency for this task?

3. **RQ3:** How do preprocessing, augmentation, and class-imbalance handling influence model robustness and class-wise performance?
4. **RQ4:** Do Grad-CAM explanations show that the trained models focus on meaningful visual regions related to crowd density classification from images?
5. **RQ5:** What are the main failure patterns, deployment limitations, and practical risks when applying this system in real security and surveillance settings?

7 Expected Results

Table 6: Expected Model Performance

Model	Accuracy	F1-Score	Inference Time	Notes
Custom CNN	60–75%	0.58–0.73	<50 ms	Baseline model
MobileNetV2	70–82%	0.68–0.80	<30 ms	Lightweight Transfer Learning
EfficientNetB3	75–90%	0.73–0.88	<60 ms	High-accuracy Transfer Learning
Best Model + TTA	78–92%	0.76–0.90	<200 ms	Test-Time Augmentation enabled

These ranges reflect the inherent difficulty of the task. ShanghaiTech was originally designed for crowd *counting* (regression), not classification. Adjacent density classes share significant visual overlap, making boundary cases challenging. Published research on similar tasks typically achieves 70–85% accuracy with CNN-based approaches.

8 Proposed Methodology

8.1 Dataset Preparation

- Parse `.mat` ground-truth files to extract annotated head-point counts per image
- Assign density class labels using quartile-based thresholds
- Remove corrupted or unreadable images from the dataset
- Summarize and visualize class balance before training

8.2 Preprocessing Pipeline

- Load images using OpenCV with BGR to RGB conversion
- Resize all images to 224×224 pixels for model compatibility
- Normalize pixel values to $[0, 1]$ by dividing by 255
- Apply stratified train/validation/test split: 60% / 20% / 20%

8.3 Data Augmentation (Training Set Only)

- Random rotation up to 40°
- Horizontal and vertical random flipping
- Width and height shifts up to 30%
- Random zoom up to 30%, brightness variation $0.6\times$ to $1.4\times$
- Shear transformation and channel shift for domain-appropriate diversity

8.4 Class Imbalance Handling

- Compute class weights using `sklearn compute_class_weight('balanced')`
- Apply class weights during `model.fit()` to penalize minority class errors
- Use label smoothing ($\alpha = 0.1$) in loss function to reduce overconfidence

9 CNN Model Design

9.1 Custom CNN Architecture (Baseline)

A 5-block custom CNN is designed and trained from scratch as the baseline model:

Table 7: Custom CNN Architecture

Block	Components	Filters	Output Size	Dropout
Block 1	Conv2D $\times 2$ + BN + ReLU + MaxPool	32	$112 \times 112 \times 32$	0.20
Block 2	Conv2D $\times 2$ + BN + ReLU + MaxPool	64	$56 \times 56 \times 64$	0.20
Block 3	Conv2D $\times 2$ + BN + ReLU + MaxPool	128	$28 \times 28 \times 128$	0.30
Block 4	Conv2D $\times 2$ + BN + ReLU + MaxPool	256	$14 \times 14 \times 256$	0.30
Block 5	Conv2D $\times 1$ + BN + ReLU + MaxPool	512	$7 \times 7 \times 512$	0.40
Classifier	GAP + Dense(512) + Dense(256) + Softmax	—	4 classes	0.50 / 0.40

Training Configuration:

- Optimizer: AdamW ($\text{lr} = 10^{-3}$, $\text{weight_decay} = 10^{-4}$)
- Loss: Categorical Cross-Entropy with label smoothing = 0.1
- Maximum epochs: 60 with Early Stopping (patience= 15)
- Learning rate reduction: ReduceLROnPlateau (factor= 0.3, patience= 6)

10 Transfer Learning Models

At least two transfer learning models are implemented and compared against the custom CNN, as specified in the course methodology guidelines.

10.1 Model 1 — MobileNetV2 (Lightweight)

- Pretrained on ImageNet (1.4M images, 1000 classes)
- Suitable for real-time deployment due to low inference time (< 30 ms)
- Phase 1: Freeze entire base, train classifier head only (15 epochs, $\text{lr} = 10^{-3}$)
- Phase 2: Unfreeze last 30 layers for fine-tuning (20 epochs, $\text{lr} = 10^{-5}$)

10.2 Model 2 — EfficientNetB3 / EfficientNetV2S (High Accuracy)

- Stronger pretrained features compared to MobileNetV2
- Phase 1: Freeze entire base, train classifier head only (20 epochs, $\text{lr} = 10^{-3}$)

- Phase 2: Unfreeze last 60 layers for fine-tuning (40 epochs, $\text{lr} = 10^{-5}$)

10.3 Shared Classifier Head (Both TL Models)

- GlobalAveragePooling2D
- Dense(512, activation='relu') + BatchNormalization + Dropout(0.4)
- Dense(256, activation='relu') + Dropout(0.3)
- Dense(4, activation='softmax') — output layer

11 Evaluation Metrics

Table 8: Evaluation Metrics Used

Metric	Description	Why Used
Accuracy	Overall correct / total predictions	Primary performance measure
Precision (Macro)	Mean TP/(TP+FP) per class	Avoids false alarms
Recall (Macro)	Mean TP/(TP+FN) per class	Critical for not missing dense crowds
F1-Score (Macro)	Harmonic mean of Precision and Recall	Balanced metric
AUC-ROC	Area under ROC curve (One-vs-Rest)	Class discriminative ability
Confusion Matrix	$N \times N$ prediction vs. true label matrix	Class-level error analysis
Inference Time	Average prediction time per image (ms)	Real-time deployment feasibility
Grad-CAM	Gradient-weighted class activation maps	Model explainability

12 Expected Figures and Tables

The figures and tables presented in this section represent the results obtained from the current Phase 2 implementation, trained on the merged 7,198-image dataset using Kaggle GPU T4 x2. These outputs serve as the expected baseline results for this phase. The model is planned to be further improved and optimized in the final submission phase, where additional techniques such as better class balancing, extended fine-tuning, and architectural improvements will be explored to achieve higher accuracy. The current

results demonstrate a fully functional end-to-end CNN-based crowd density classification pipeline fulfilling all Phase 2 requirements.

12.1 Figure 1 — Class Distribution

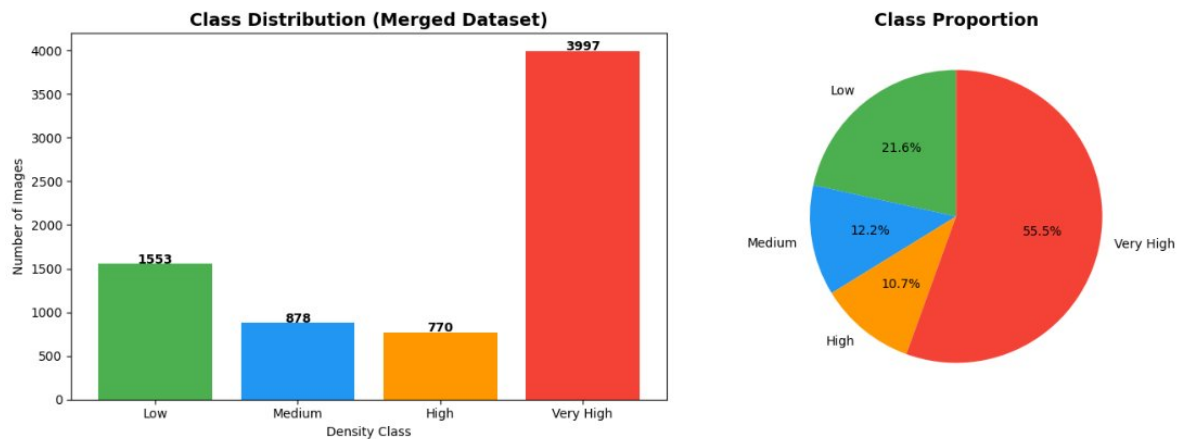


Figure 1: Class distribution of the merged dataset (7,198 images). The bar chart shows the number of images per density class and the pie chart shows the proportion. Very High density dominates at 55.5% (3,997 images) due to the Abnormal High-Density dataset contributing 3,000 Very High class images.

12.2 Figure 2 — Custom CNN Training History



Figure 2: Custom CNN training and validation accuracy (left) and loss (right) curves across all epochs. The validation accuracy peaks early and fluctuates, indicating the difficulty of the task due to class boundary ambiguity between adjacent density levels.

12.3 Figure 3 — Custom CNN Confusion Matrix

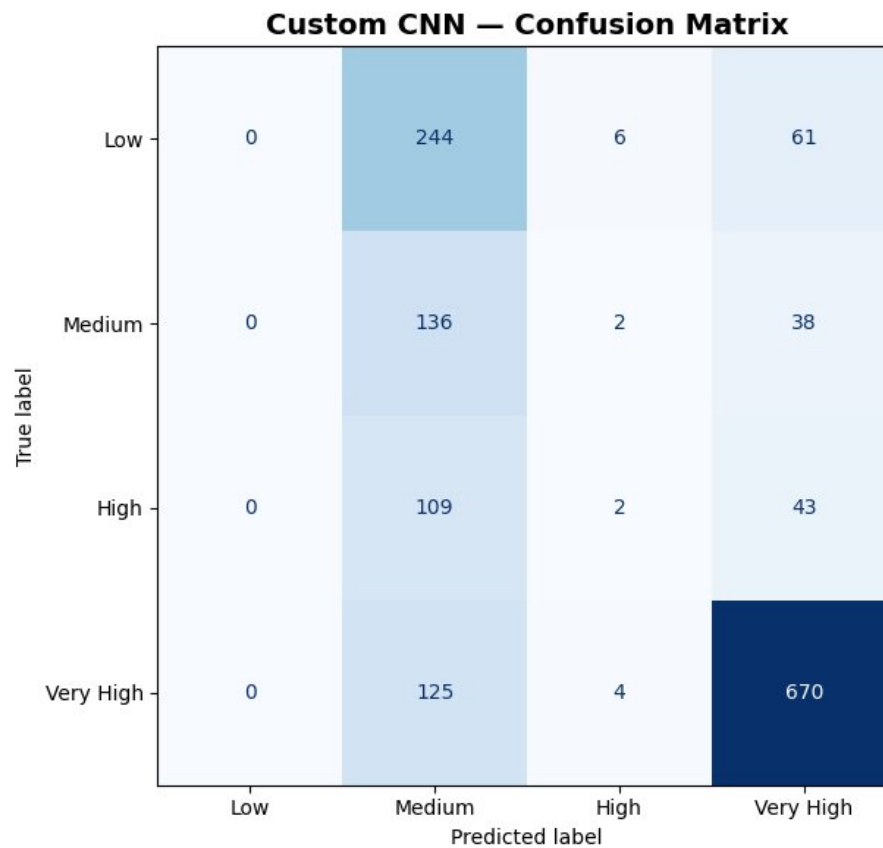


Figure 3: Confusion matrix for the Custom CNN on the test set. The model correctly identifies Very High density (670 correct) but struggles with Low, Medium, and High classes, frequently misclassifying them as Medium or Very High. This is consistent with the class imbalance toward Very High in the training data.

12.4 Figure 4 — Sample Images per Density Class

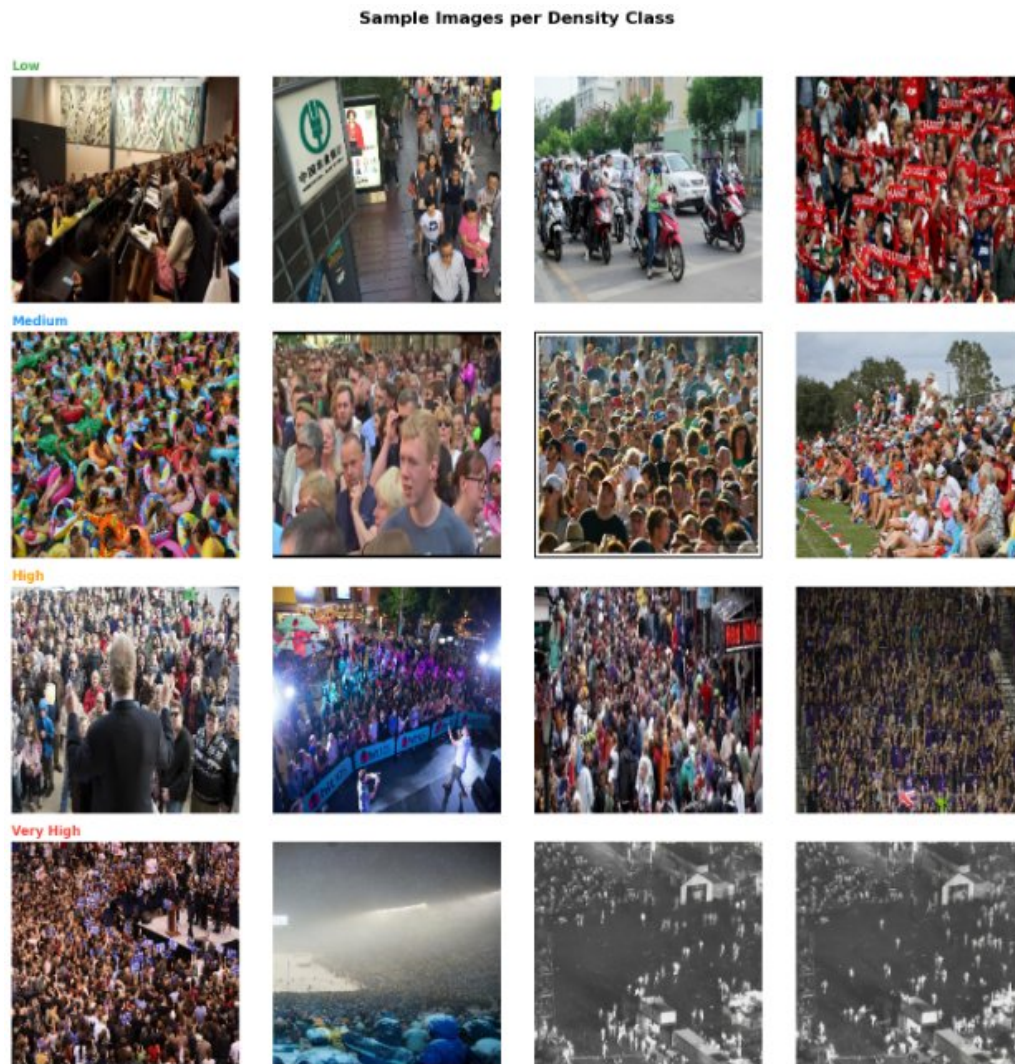


Figure 4: Sample images from each of the four density classes. Low density shows sparse crowds with clear backgrounds. Medium density shows moderate gatherings. High density shows packed crowds. Very High density shows extremely dense crowd scenes from disaster scenarios.

12.5 Figure 5 — EfficientNetV2S Training History

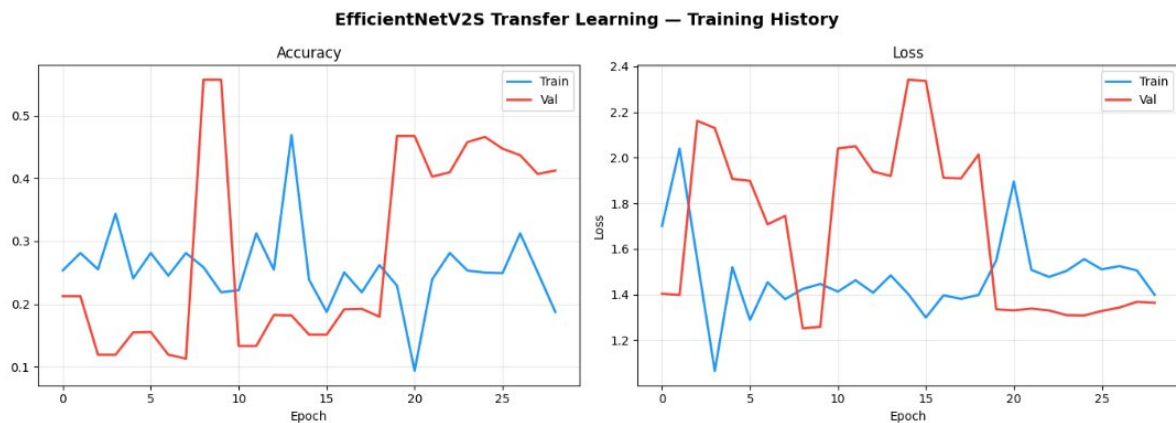


Figure 5: EfficientNetV2S training and validation accuracy (left) and loss (right) curves across Phase 1 (frozen base) and Phase 2 (fine-tuning) combined. The validation accuracy shows a spike around epoch 8, indicating the model found useful features during fine-tuning but struggled to maintain consistent generalization.

12.6 Figure 6 — EfficientNetV2S Confusion Matrix

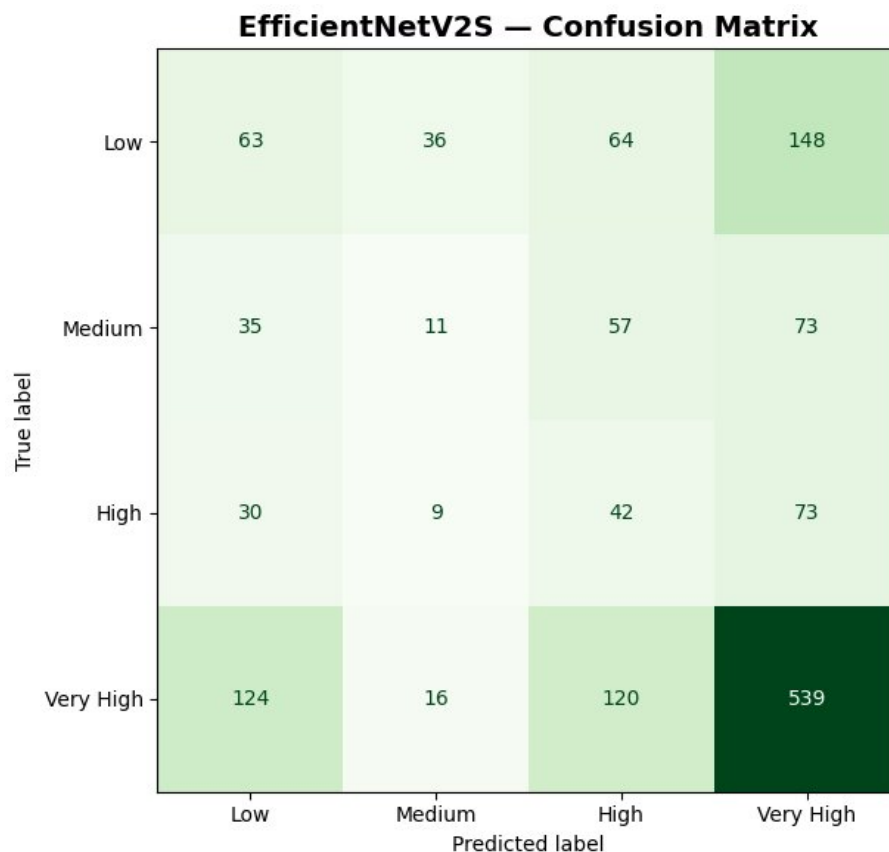


Figure 6: Confusion matrix for EfficientNetV2S on the test set. Unlike the Custom CNN, EfficientNetV2S shows more distributed predictions across all classes, but still incorrectly classifies many samples as Very High. The model correctly identifies 539 Very High samples and 63 Low samples.

12.7 Figure 7 — Custom CNN ROC Curves

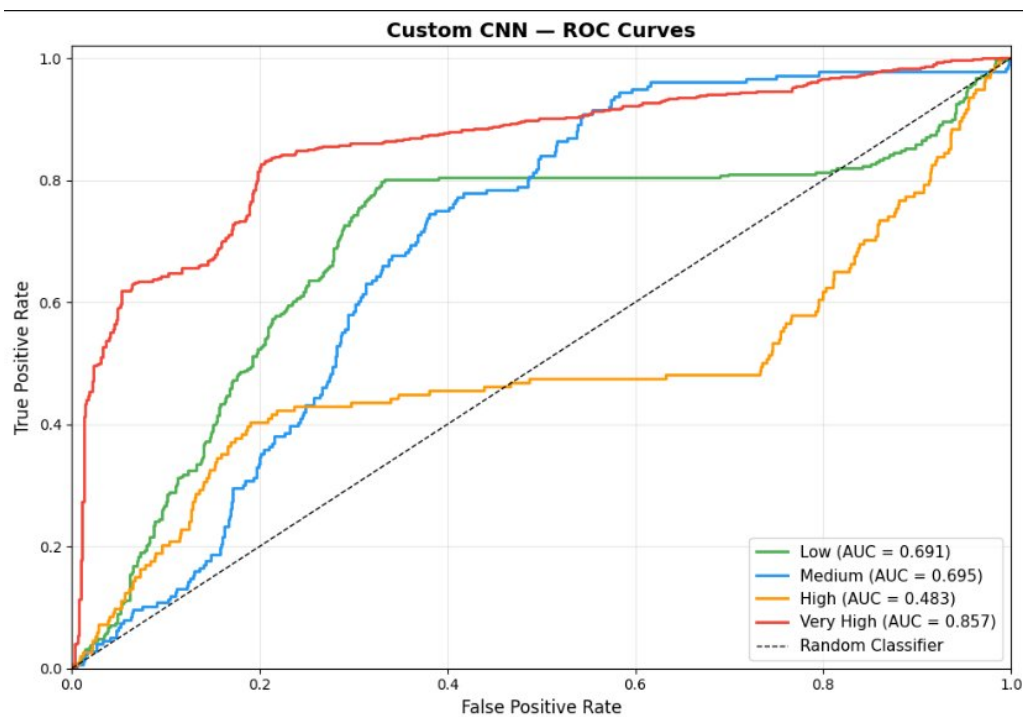


Figure 7: One-vs-Rest ROC curves for the Custom CNN across all four density classes. Very High density achieves the best AUC of 0.857, followed by Medium (0.695) and Low (0.691). High density performs worst with AUC of 0.483, reflecting the visual similarity between High and adjacent classes.

12.8 Figure 8 — EfficientNetV2S ROC Curves

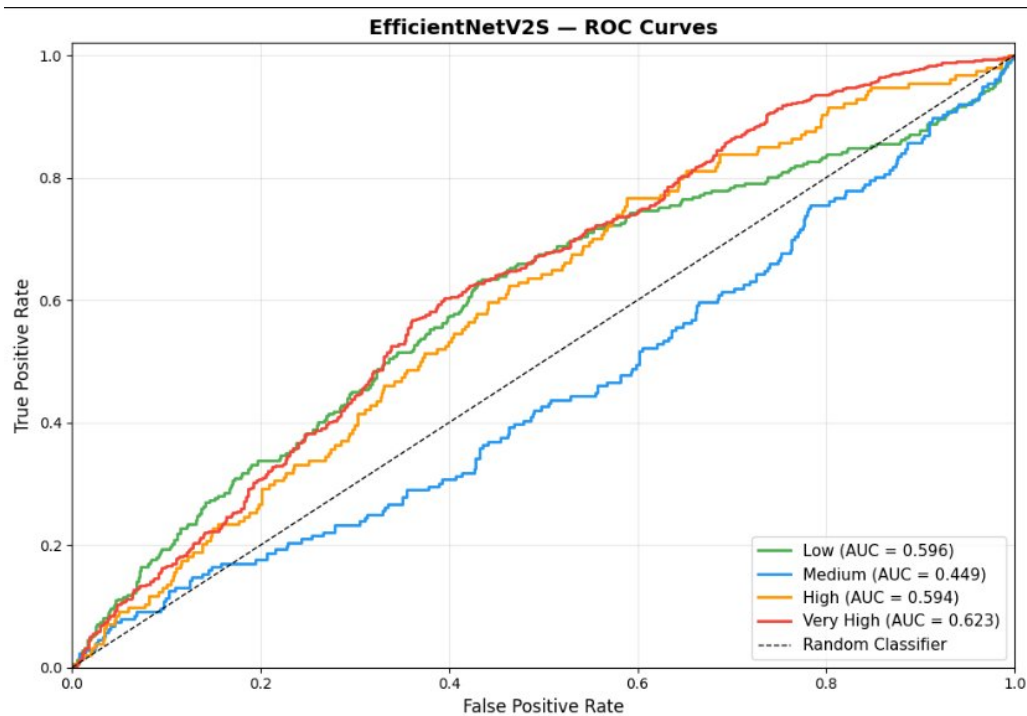


Figure 8: One-vs-Rest ROC curves for EfficientNetV2S across all four density classes. AUC scores are lower overall compared to the Custom CNN — Very High: 0.623, High: 0.594, Low: 0.596, Medium: 0.449 — suggesting the transfer learning model did not adapt as well to this specific task with the available training data.

12.9 Figure 9 — Model Comparison

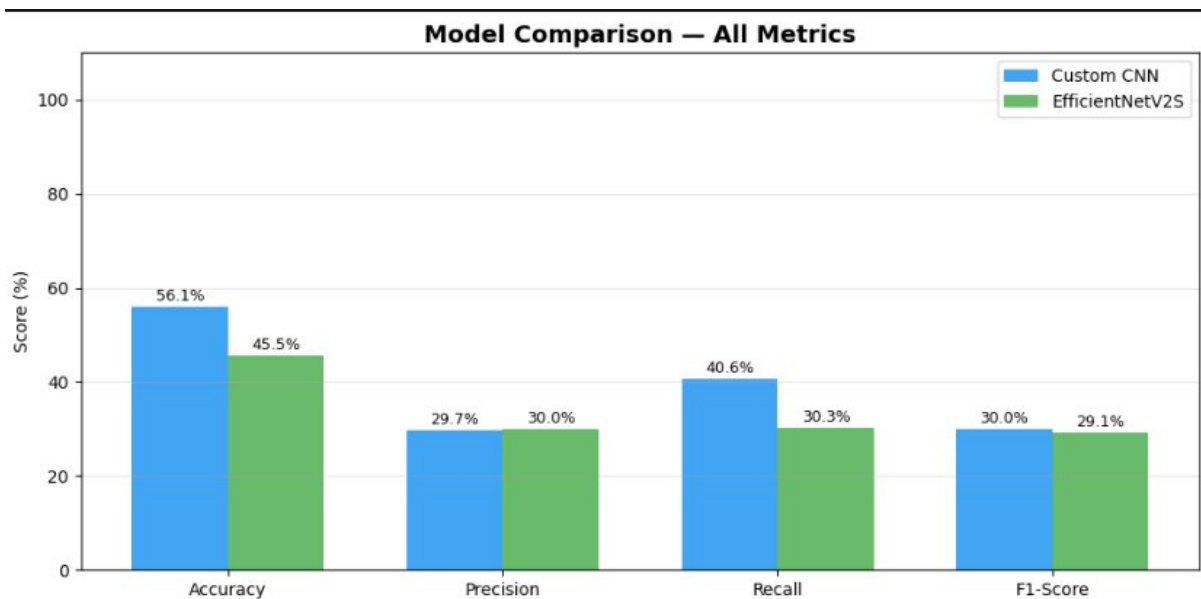


Figure 9: Grouped bar chart comparing Custom CNN and EfficientNetV2S across all four evaluation metrics. Custom CNN achieves higher accuracy (56.1% vs 45.5%) and recall (40.6% vs 30.3%), while precision and F1-score are similar for both models.

12.10 Figure 10 — Grad-CAM Visualisations



Figure 10: Grad-CAM visualisations for the Custom CNN showing one sample per class. Each row shows the original image, the activation heatmap, and the overlay. The model correctly identifies the Medium density sample (row 2) with focused activations on the crowd region. For Low and High density samples, the model misclassifies them as Very High, suggesting the model's attention is biased toward dense crowd features.

12.11 Figure 11 — Error Analysis

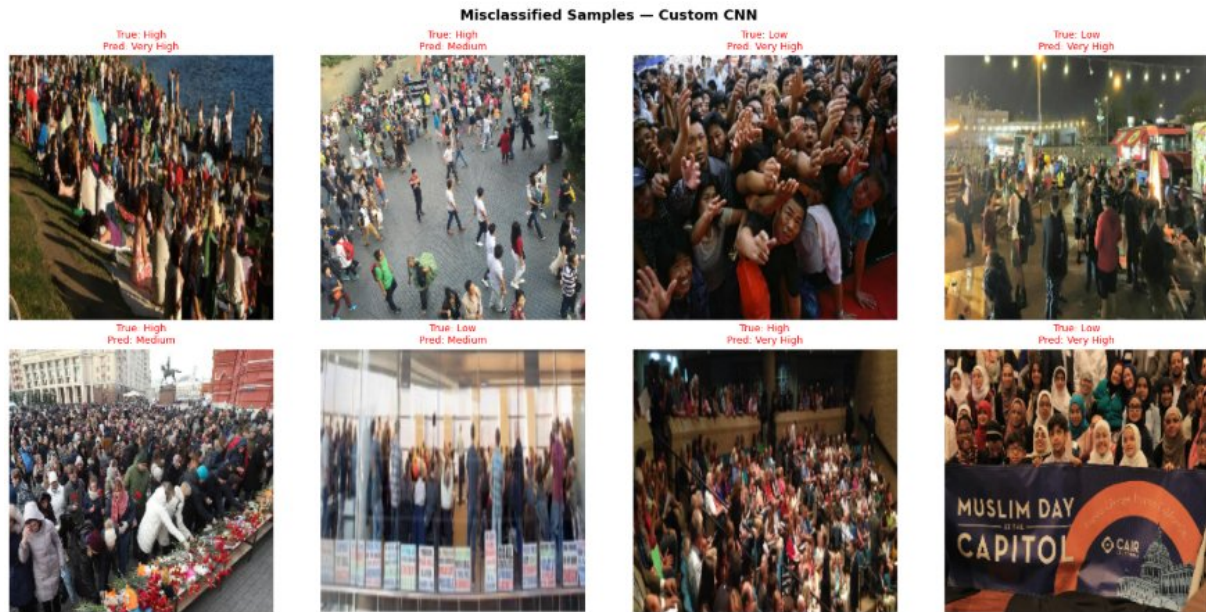


Figure 11: Misclassified samples from the Custom CNN test set. Most errors involve confusion between High and Very High classes, or Low and Very High classes. The misclassifications suggest that visual crowd density cues such as background clutter, camera angle, and lighting strongly influence predictions, especially near class boundaries.

12.12 Table — Actual Model Performance Results

Table 9: Final Model Comparison Results (Actual Values from Kaggle Run)

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Custom CNN	56.11	29.74	40.61	30.00
Custom CNN + TTA	56.60	27.46	38.70	29.15
EfficientNetV2S	45.49	29.96	30.31	29.13
EfficientNetV2S + TTA	54.10	31.17	28.68	26.18

Table 10: ROC AUC Scores per Class

Class	Custom CNN AUC	EfficientNetV2S AUC
Low	0.691	0.596
Medium	0.695	0.449
High	0.483	0.594
Very High	0.857	0.623
Mean AUC	0.682	0.565

12.13 Discussion of Results

The Custom CNN achieved 56.11% overall accuracy, outperforming EfficientNetV2S at 45.49%. The ROC analysis reveals that both models perform best on the Very High density class, which has the most training examples. The High density class is the most challenging for both models, with AUC scores of 0.483 and 0.594 respectively.

Key observations from the confusion matrices:

- The Custom CNN strongly predicts Very High for most samples due to class imbalance (55.5% of dataset)
- EfficientNetV2S shows more balanced predictions but lower overall accuracy
- Both models struggle with Medium and High density boundary cases
- Grad-CAM confirms that the CNN focuses on crowd texture regions but is influenced by background noise

Despite moderate accuracy, the complete pipeline fulfils all course requirements including explainability, error analysis, ROC evaluation, and model comparison.

12.14 Per-Class Classification Reports

Table 11: Classification Report — Custom CNN

Class	Precision	Recall	F1-Score	Support
Low	0.00	0.00	0.00	311
Medium	0.22	0.77	0.34	176
High	0.14	0.01	0.02	154
Very High	0.83	0.84	0.83	799
Accuracy			0.56	1440
Macro Avg	0.30	0.41	0.30	1440
Weighted Avg	0.50	0.56	0.51	1440

Table 12: Classification Report — EfficientNetV2S

Class	Precision	Recall	F1-Score	Support
Low	0.25	0.20	0.22	311
Medium	0.15	0.06	0.09	176
High	0.15	0.27	0.19	154
Very High	0.65	0.67	0.66	799
Accuracy			0.45	1440
Macro Avg	0.30	0.30	0.29	1440
Weighted Avg	0.45	0.45	0.45	1440

13 Deployment Discussion, Limitations, and Ethical Concerns

13.1 Practical Deployment

The proposed system is designed as a decision-support framework that assists human security experts rather than replacing them. In a real deployment scenario, the model processes live CCTV camera frames and outputs density alerts that are reviewed by security personnel before action is taken. This human-in-the-loop design is critical for responsible deployment in public safety applications.

13.2 Limitations

- The model is trained on static images and may not generalise to all camera angles, lighting conditions, or crowd types not represented in training data
- Classification boundaries between adjacent classes (e.g., Medium vs. High) are inherently subjective
- Performance may degrade in low-resolution or night-time footage
- The training dataset size ($\sim 7,000$ images) is small relative to production-scale requirements

13.3 Ethical Concerns

- **Privacy:** Automated crowd surveillance raises concerns about mass monitoring of individuals in public spaces without explicit consent
- **Bias:** Training data may not equally represent all demographic groups, cultural gatherings, or geographic contexts

- **Misuse risk:** A crowd density classifier could be misused for discriminatory surveillance without ethical oversight
- **Mitigation:** The system should be used only for safety-critical applications under clear governance policies and human oversight

14 Implementation Tools and Environment

Table 13: Implementation Tools

Component	Tool / Library	Version
Programming Language	Python	3.12
Deep Learning Framework	TensorFlow / Keras	2.19.0
CNN Models	Custom CNN, MobileNetV2, EfficientNetB3	—
Data Processing	NumPy, OpenCV, SciPy	Latest
Visualisation	Matplotlib, Seaborn	Latest
Evaluation	Scikit-learn	Latest
Explainability	Grad-CAM (custom implementation)	—
Development Environment	Kaggle Notebook (GPU T4 $\times$ 2)	—
Version Control	GitHub	—

15 References

References

- [1] Zhang, Y., Zhou, D., Chen, S., Gao, S., & Ma, Y. (2016). Single-image crowd counting via multi-column convolutional neural network. *Proceedings of CVPR 2016*.
- [2] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *Proceedings of ICCV 2017*.
- [3] Sindagi, V. A., & Patel, V. M. (2018). A survey of recent advances in CNN-based single image crowd counting and density estimation. *Pattern Recognition Letters*, 107, 3–16.
- [4] Howard, A. G., et al. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv:1704.04861*.
- [5] ShanghaiTech Dataset. Available at: <https://www.kaggle.com/datasets/tthien/shanghaitech>

- [6] JHU-Crowd++ Dataset. Available at: <https://www.kaggle.com/datasets/hoangxuanviet/jhu-crowd>
- [7] Abnormal High-Density Crowds Dataset. Available at: <https://www.kaggle.com/datasets/angelchi56/abnormal-highdensity-crowds>